CS 231N · SPRING 2026 · PROJECT PROPOSAL

MILESTONE 1 — PROBLEM + RELATED WORK

Scaling Drifting VQ-VAE *to Real Images*

Can a physics-inspired fix for codebook collapse survive the jump from 2D toys to deep CNNs on natural images?

Ethan Zhang ez26@stanford.edu
Hemal Arora hemal1@stanford.edu
Tesvara Jiang tesvaraj@stanford.edu

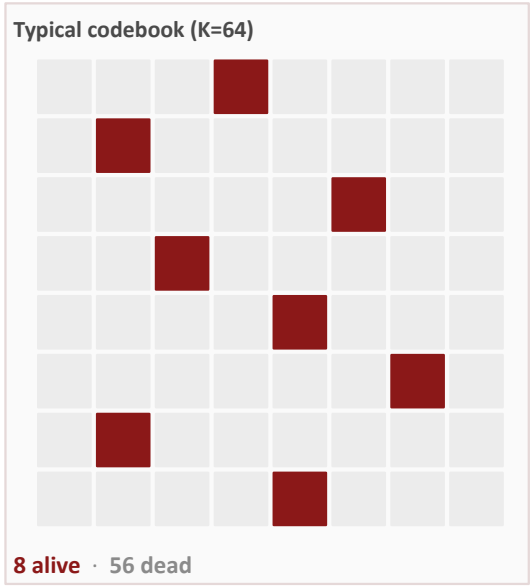
The problem: codebook collapse

Why VQ-VAEs matter

VQ-VAEs compress images into sequences of discrete tokens from a fixed codebook. This idea underlies many image/video tokenization pipelines, including VQGAN, DALL-E-style discrete image modeling, and VideoGPT.

The failure mode

Only the nearest codeword to each encoder output receives the direct codebook update. Codes that start far from data may rarely be selected, receive little or no useful update, and become effectively dead. In severe collapse, a large codebook can behave like a much smaller vocabulary.



Consequence: wasted model capacity, fewer effective tokens, and weaker downstream generation on top of the discrete latent space.

Related work and positioning

FOUNDATION

VQ-VAE

van den Oord et al., 2017

Introduces VQ-VAE: a variational autoencoder with discrete latent codes learned by nearest-neighbor vector quantization. Provides our vanilla VQ baseline.

Our starting baseline.

CONTEXT

VQGAN

Esser et al., 2021

Extends VQ-style image tokenization with perceptual/adversarial training and an autoregressive transformer prior, showing that learned discrete image tokens can support SOTA image synthesis.

Why robust tokenizers matter.

STRONG BASELINE

SimVQ

Zhu et al., 2025

Reparameterizes code vectors through a shared linear layer so optimization affects the whole codebook space, rather than only selected nearest-neighbor codes.

Our primary competing method.

OUR BASIS

Drifting VQ-VAE

Liu, 2026 (Blog Post)

Treats encoder outputs (+) and codebook entries (−) as charges; a potential-energy term encourages coverage and spread. This was demonstrated only in a toy setting; we test whether it scales to real images.

Toy 2D only, but we will attempt scaling to real images.

Physics-inspired forces on the latent space

How it works

Encoder outputs **+** · Codebook entries **−**

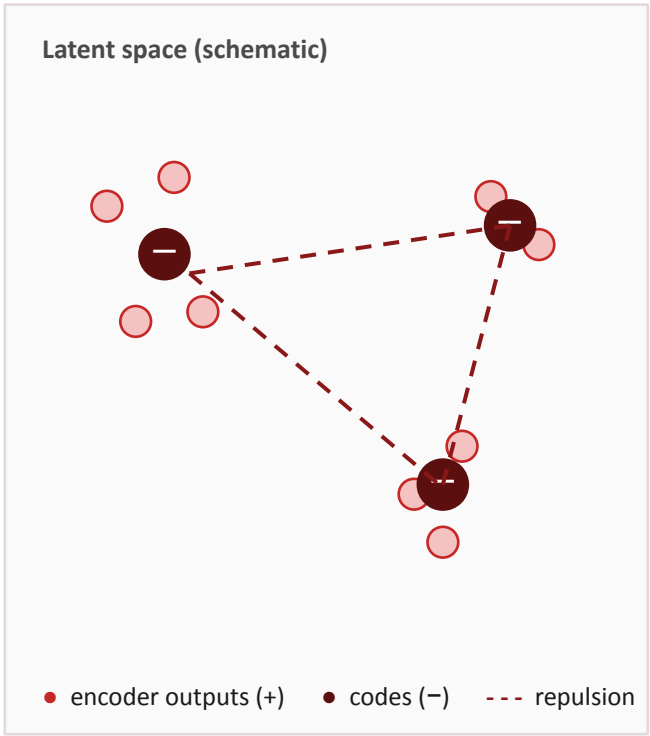
Attraction (+,−): Codes receive force toward encoder outputs, giving unused codes a path to move toward data.

Repulsion (−,−)/(+,+): Same-sign particles repel, discouraging encoder outputs or codes from collapsing onto only a few locations. In sum, the total positive charge is +1 and the total negative charge is -1, making the system electrically neutral overall, so equilibrium is stable.

Every encoder output is treated as a positive charge, and every codebook entry as a negative charge. Opposite charges attract, pulling codes toward the encoded data distribution. Since repulsion encourages spread, assignments are less likely to pile onto only a few codes compared to the original VQ-VAE.

OUR HYPOTHESIS

The same +/− potential that solved Liu's 2D toy case will improve codebook utilization while preserving competitive reconstruction quality when scaled to deep CNN encoders on CIFAR-10 and CelebA.



Setup and expected outcomes

We will take Liu's drifting potential, plug it into a real CNN VQ-VAE pipeline, train it on CIFAR-10 and CelebA, and see what happens.

Experimental setup

Datasets	CIFAR-10 · CelebA
Architecture	Shared CNN encoder + decoder across methods
Methods	Vanilla VQ-VAE · SimVQ · Drifting VQ-VAE
Codebook sizes	K = 64, 256, 512, 1024
Metrics	Codebook utilization · perplexity · PSNR · SSIM · FID · loss curves

Three potential outcomes

BEST

Clean win

Drifting improves utilization/perplexity over SimVQ at large K while matching or improving reconstruction/FID.

MIDDLE

Partial advantage

Drifting is competitive overall, with a clearer advantage at small K, during early training, or in stability.

NULL

Negative result

Drifting does not improve utilization or hurts reconstruction quality when scaled to real images, or both.

Success criterion: A quantitative answer with utilization/perplexity plots over K to whether Drifting VQ-VAE survives the toy-to-real gap.

05 — ALTERNATIVES

Alternative solutions to VQ collapse problem

There are methods that sidestep learnable VQ collapse, but they do not replace discrete learned tokenizers in every setting.

ALTERNATIVE · 2023

Finite Scalar Quantization

Mentzer et al., ICLR 2024

Replaces learned vector quantization with a fixed scalar quantizer: project the latent to a few dimensions, bound each dimension, then round each coordinate to one of a small set of fixed values. The implicit codebook is the Cartesian product of those scalar levels.

THE CATCH

FSQ avoids codebook collapse by removing the learned vector codebook; it does not tell us whether learnable VQ itself can be stabilized.

ALTERNATIVE · 2022

Continuous Latent Diffusion

Rombach et al., 2022; Stable Diffusion family

Skips discrete token prediction and trains diffusion models in the continuous latent space of a pre-trained autoencoder, reducing compute while preserving high-quality synthesis.

THE CATCH

Continuous latents are powerful for diffusion, but they are not discrete learned visual tokens for language-model-style sequence generation.

WHY LEARNABLE VQ STILL MATTERS

FSQ and latent diffusion avoid collapse by changing the bottleneck; but because learned discrete codes remain useful for transformer-style image, video, audio, and multimodal generation, our project asks whether VQ can be stabilized using physics rather than replaced.

Appendix

Problem setup

VQVAE

VQVAE consists of an encoder E, a decoder D, and K codes in the latent space. The encoder maps the input x to a hidden state $h = E(x)$. The hidden state is then mapped to the nearest code $e = \mathbf{argmin}_i \|h - e_i\|$. The loss consists of three parts (where sg means stop gradients):

- Reconstruction loss $\ell_{\text{rec}} = \|x - D(e)\|^2$
- Codebook loss $\ell_{\text{code}} = \|\mathbf{sg}[E(x)] - e\|^2$
- Commitment loss $\ell_{\text{commit}} = \|E(x) - \mathbf{sg}[e]\|^2$

The total loss is $\ell = \ell_{\text{rec}} + \ell_{\text{code}} + \beta \ell_{\text{commit}}$. Typically $\beta = \mathbf{0.25}$ is a good choice.

Dataset

The dataset is intentionally simple: two clusters in the 2D plane.

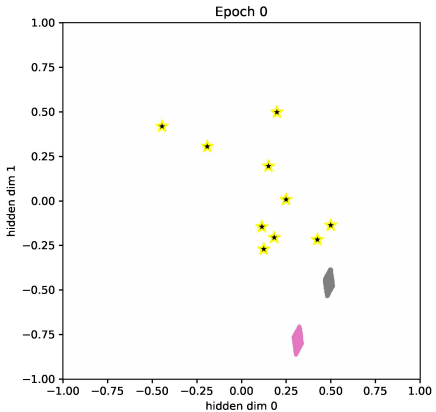
Model

Both the encoder and decoder are linear layers $\mathbb{R}^2 \to \mathbb{R}^2$.

This is the simplest setup I could imagine. Given its simplicity, one might expect that $K = \mathbf{2}$ should suffice.

Failure mode of VQVAE

We visualize the hidden space during training. Codewords are visualized as stars. And the data points are visualized as blue points (they are very dense, so they look more like a rectangle or a diamond).



Drifting VQ-VAE: Why drifting models can map?

Concepts

If we interpret hidden states as positive charges and codewords as negative charges, the matching problem can be naturally solved by "drifting models". Drifting models ensure that positive charges fall onto negative charges in the correct proportion — roughly one positive charge per negative charge. Situations where multiple positive charges collapse onto a single negative charge are naturally avoided, which prevents dead codes.

Note that when I refer to "drifting models," I do not necessarily mean generative models themselves, but rather the general attraction–repulsion formulation. See my previous blogpost for a physical interpretation of drifting models.

Experiments

There are N data points — so we assign a positive charge $q = 1/N$ to each data point. There are K codewords — so we assign a negative charge $q = -1/K$ to each codeword.

The potential energy between any two particles is defined as

$$U = \tau(r + \tau)\exp^{-r/\tau}$$

(corresponding to force $f(r) = r\exp(-r/\tau)$, which is a standard choice in drifting models).

The total potential energy is therefore

$$U = U_{pp} + U_{pn} + U_{nn}$$

and the loss function becomes

$\ell = \ell_{\text{rec}} + \beta U$.

I call this method **Drifting VQ-VAE**.